Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 8
по дисциплине «Типы и структуры данных»

Тема Задача коммивояжёра

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Волкова Л.Л.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Задача коммивояжёра 5

1.2 Алгоритм полного перебора 5

1.3 Алгоритм муравья 5

2 Конструкторская часть **7**

2.1 Структура Graph 7

2.2 Структура Ant 7

2.3 Алгоритм муравья 7

2.4 Алгоритм полного перебора 9

3 Технологическая часть **10**

3.1 Реализация и выбор средств 10

3.2 Тестирование программы 10

4 Исследовательская часть **13**

4.1 Оценка трудоемкости алгоритмов 13

4.2 Сравнительный анализ алгоритмов 14

4.3 Параметризация 15

ЗАКЛЮЧЕНИЕ **17**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **18**

ПРИЛОЖЕНИЕ А **19**

ПРИЛОЖЕНИЕ В **26**

ВВЕДЕНИЕ

Целью работы является описание и реализация решения задачи коммивояжёра с помощью **алгоритма полного перебора** и **алгоритма муравья**. Для осуществления поставленной задачи необходимо выполнить следующие пункты:

1) описать методы решения задачи коммивояжёра на основе алгоритма полного перебора и алгоритма муравья в соответствии с заданием индивидуального варианта;
2) описать данные алгоритмы схемами;
3) на основе составленных схем выполнить оценку трудоемкости алгоритмов;
4) реализовать описанные алгоритмы;
5) выполнить параметризацию второго метода;
6) провести сравнительный анализ описанных алгоритмов.

Задание индивидуального варианта: неориентированный граф, без элитных муравьёв, карта городов древнего мира, гамильтонов цикл.

1 Аналитическая часть

1.1 Задача коммивояжёра

Задача коммивояжёра — это классическая NP-трудная задача комбинаторной оптимизации. Она заключается в следующем: дан граф $G = (V, E)$, где V — множество вершин (города), а E — множество рёбер с весами (стоимостью перемещения между городами), и требуется найти маршрут минимальной стоимости, который проходит через каждую вершину ровно один раз.

1.2 Алгоритм полного перебора

Полный перебор предполагает проверку всех возможных перестановок вершин графа G для нахождения маршрута минимальной стоимости. Для каждого пути вычисляется суммарная стоимость и выбирается минимальная. Данный метод работает для любых ориентированных графов и гарантированно находит точное оптимальное решение, так как проверяются все возможные маршруты, однако время выполнения растёт факториально с увеличением числа вершин $O(n!)$, что делает алгоритм практически непригодным для задач с большим числом городов.

1.3 Алгоритм муравья

Муравьиный алгоритм — это один из методов решения комбинаторных задач оптимизации, в том числе задачи коммивояжёра. Он основан на моделировании поведения колонии муравьёв при поиске кратчайшего пути между источником пищи и муравейником и подходит для сложных комбинаторных задач [1].

Основная идея алгоритма заключается в использовании положительной обратной связи через обновление феромонов на рёбрах графа, что позволяет усилить предпочтения более коротких маршрутов. Алгоритм относится к эвристическим методам и не гарантирует нахождения глобального оптимума, но может предоставить близкие к оптимальным решения, а эффективность работы алгоритма сильно зависит от правильного подбора параметров (коэффициенты испарения феромона, влияние эвристики и феромонов). Далее приведено описание работы данного алгоритма.

Вес ребра $d_{i,j}$ между вершинами i и j задаёт расстояние между ними. Пусть τ_{ij} — интенсивность феромона на ребре (i, j), а $\eta_{ij} = \dfrac{1}{d_{ij}}$ — эвристическая информация, обратно пропорциональная длине пути. Ниже приведены основные шаги алгоритма муравья.

1. **Инициализация**. Феромоны на всех рёбрах графа инициализируются одинаковым значением τ_0. Определяется количество муравьёв m, параметр испарения феромона $\rho \in (0, 1)$ и другие параметры алгоритма. Задаётся константное значение Q, которое

вычисляется перед началом алгоритма по формуле:

$$Q = \frac{\sum_{i=1}^{N} \sum_{j=1}^{N} d_{ij}}{N-1}, \tag{1.1}$$

где N — количество городов.

2. **Построение маршрутов**. Каждый муравей начинает путь из случайной вершины графа и поэтапно переходит в следующую вершину, пока не посетит все вершины. Переход из вершины i в вершину j осуществляется с вероятностью $P_{ij}(t)$, которая рассчитывается по формуле:

$$P_{ij}(t) = \begin{cases} \dfrac{[\tau_{ij}(t)]^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum\limits_{k \in \text{доступные}} [\tau_{ik}(t)]^{\alpha} \cdot \eta_{ik}^{\beta}}, & \text{если } j \text{ доступна;} \\ 0, & \text{иначе} \end{cases} \tag{1.2}$$

где α и β — параметры, определяющие влияние феромонов и эвристической информации соответственно ($\alpha \geq 0, \beta \geq 0$).

3. **Обновление феромонов**. После прохождения всех муравьёв по графу феромоны на рёбрах обновляются по формуле:

$$\tau_{ij}(t+1) = (1-\rho) \cdot \tau_{ij}(t) + \sum_{k=1}^{m} \Delta\tau_{ij}^{k}, \tag{1.3}$$

где $\rho \in (0,1)$ — коэффициент испарения феромона, а $\Delta\tau_{ij}^{k}$ — количество феромона, оставленного k-м муравьём на ребре (i,j):

$$\Delta\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{L_k}, & \text{если муравей } k \text{ прошёл через ребро } (i,j); \\ 0, & \text{иначе,} \end{cases} \tag{1.4}$$

где L_k — длина пути, пройденного k-м муравьём.

4. **Критерий остановки**. Алгоритм завершается, когда достигнуто максимальное число итераций (дней).

2 Конструкторская часть

В данном разделе представлены описания необходимых структур и схемы алгоритмов муравья и полного перебора для решения задачи коммивояжёра.

2.1 Структура Graph

Структура **Graph** предназначена для хранения полногосвязного графа и содержит следующие поля:

— целое число `n` — количество вершин (городов) графа;

— массив строк `names` — названия городов, упорядоченные в соответствии с индексами вершин;

— матрица `dist` — матрица расстояний между вершинами размера $n \times n$, где `dist`$[i][j]$ — расстояние между городами i и j.

Структура используется для хранения исходного графа, на котором работают муравьиный алгоритм и алгоритм полного перебора.

2.2 Структура Ant

Структура **Ant** предназначена для хранения состояния одного муравья в муравьином алгоритме и содержит следующие поля:

— массив целых чисел `tour` — маршрут муравья, то есть последовательность номеров вершин, посещённых им;

— логический массив `visited` — посещённые города, где `visited[i]` принимает значение `true`, если город i уже включён в маршрут;

— вещественное число `length` — длина гамильтонова цикла, построенного данным муравьём.

Структура используется для представления результата обхода графа каждым муравьём и для вычисления изменения феромонов.

2.3 Алгоритм муравья

Схема алгоритма муравья представлена на рисунке 2.1.

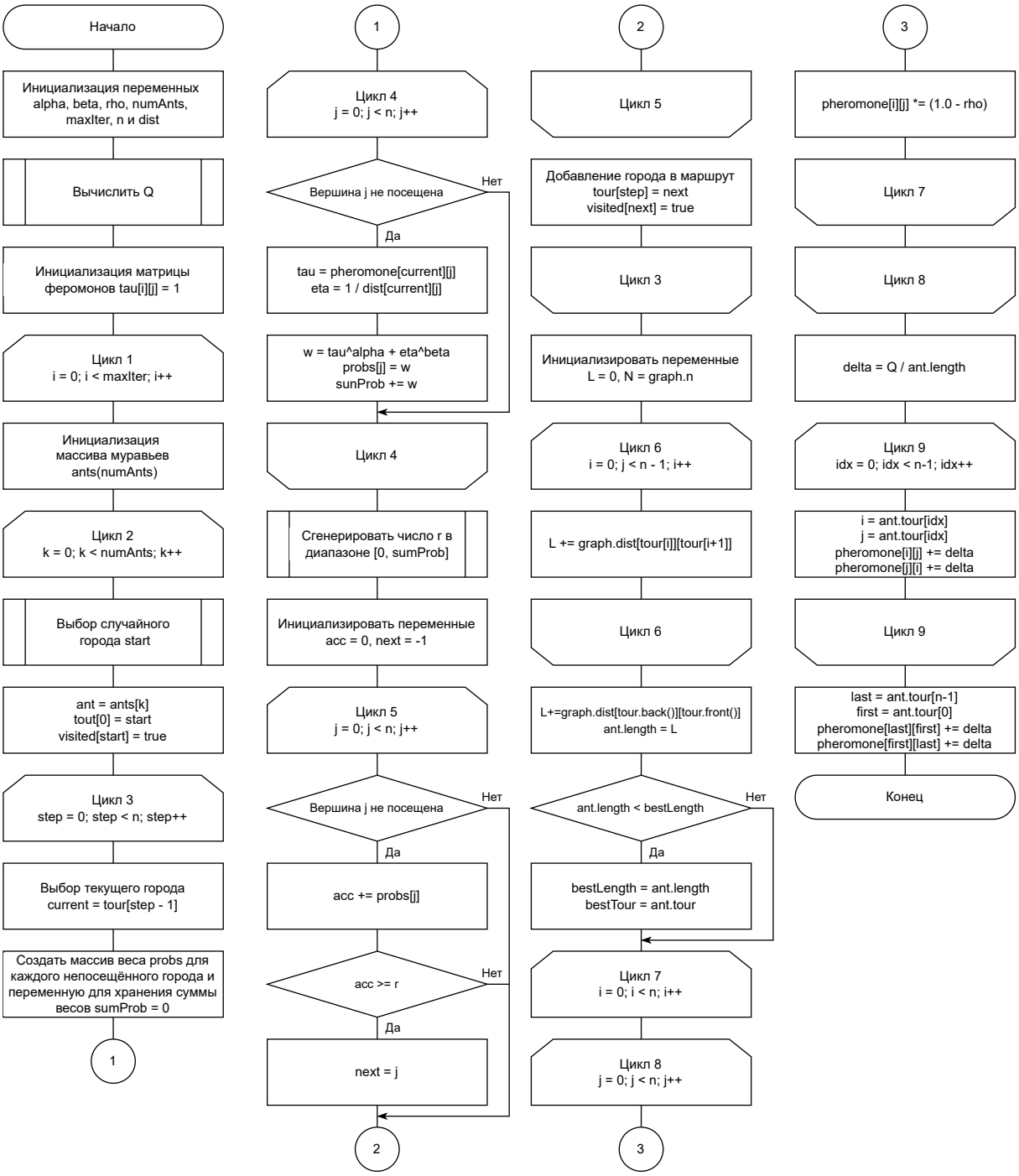

Рисунок 2.1 — Схема алгоритма муравья

2.4 Алгоритм полного перебора

Схема алгоритма полного перебора представлена на рисунке 2.2.

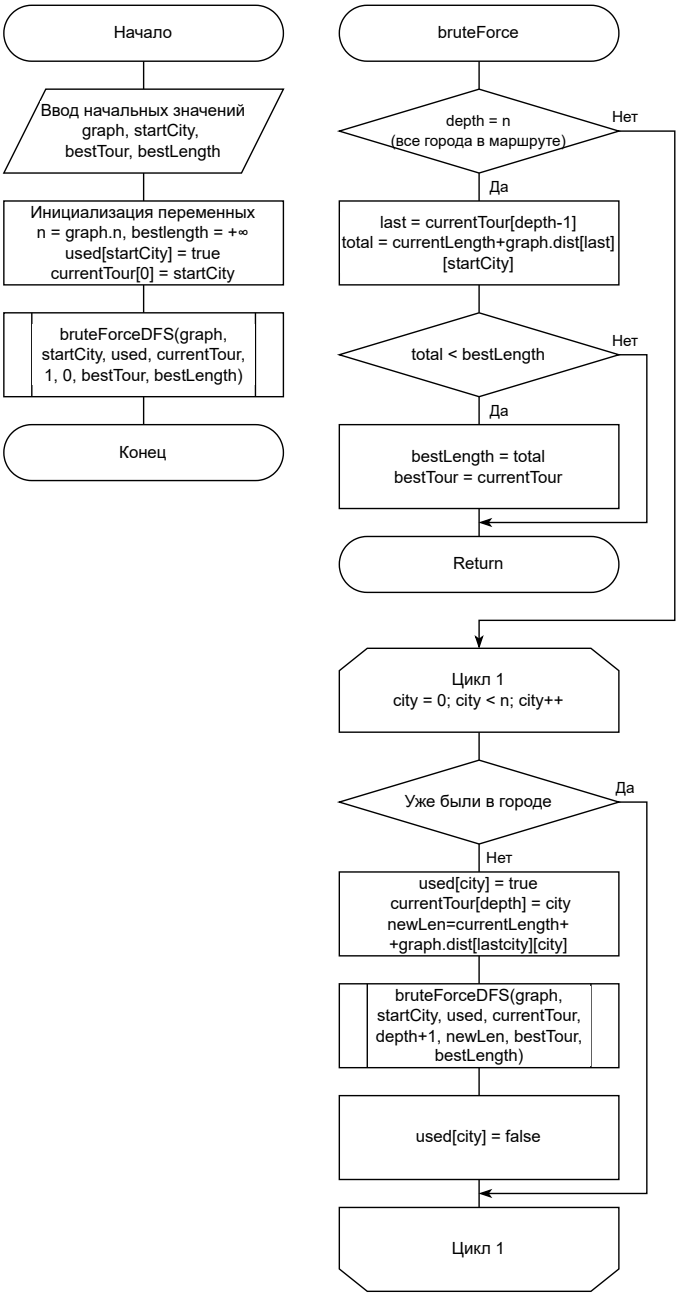

Рисунок 2.2 — Схема алгоритма полного перебора

Вывод

Были разработаны структуры **Ant** и **Graph**, а также схемами описаны алгоритмы муравья и полного перебора.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации использовалась среда разработки $Visual\ Studio\ Code$ [3]. Язык программирования, на котором была выполнена реализация алгоритмов — $C++$. Реализация программы содержит структуры и методы, описанные в предыдущем разделе. В листинге А.1 представлена программная реализация описанных алгоритмов.

3.2 Тестирование программы

Программа принимает список городов и матрицу расстояний между ними из файла. Была реализована функция `loadFromFile`, предназначенная для считывания городов и матрицы из файла и проверяющая корректность считываемых данных: элементы матрицы должны быть типа `integer`, количество строк и столбцов должно быть равным и совпадать с количеством городов. Для проверки работы реализаций описанных алгоритмов был выбран текстовый файл, содержащий в первой строке 12 городов и матрицу расстояний между ними размером 12×12. Расстояния указаны в сотнях километров. В таблице 3.1 представлены названия городов и соответствующие им номера, а в таблице 3.2 — матрица расстояний между городами, где каждому городу соответствует свой номер из таблицы 3.1.

Таблица 3.1 — Соответствие городов и их номеров

Город	**Номер**
Вавилон	1
Ниневия	2
Иерусалим	3
Тир	4
Мемфис	5
Александрия	6
Карфаген	7
Афины	8
Спарта	9
Троя	10
Византий	11
Суса	12

Таблица 3.2 — Матрица расстояний между городами (в сотнях километров)

№	1	2	3	4	5	6	7	8	9	10	11	12
1	0	3	6	7	9	10	15	13	14	11	10	4
2	3	0	5	6	10	11	16	14	15	12	11	5
3	6	5	0	1	7	8	13	12	13	11	10	7
4	7	6	1	0	7	8	12	11	12	10	9	8
5	9	10	7	7	0	2	6	7	8	9	10	11
6	10	11	8	8	2	0	5	6	7	10	11	12
7	15	16	13	12	6	5	0	8	9	12	13	17
8	13	14	12	11	7	6	8	0	1	3	4	12
9	14	15	13	12	8	7	9	1	0	4	5	13
10	11	12	11	10	9	10	12	3	4	0	2	10
11	10	11	10	9	10	11	13	4	5	2	0	9
12	4	5	7	8	11	12	17	12	13	10	9	0

В листинге 3.1 приведен результат работы программы.

```
Загружено городов: 12
Запуск муравьиного алгоритма...
Итерация 50, лучшая длина = 54 (в сотнях км)
Итерация 100, лучшая длина = 54 (в сотнях км)
Итерация 150, лучшая длина = 54 (в сотнях км)
Итерация 200, лучшая длина = 54 (в сотнях км)
Итерация 250, лучшая длина = 54 (в сотнях км)
Итерация 300, лучшая длина = 54 (в сотнях км)
Итерация 350, лучшая длина = 54 (в сотнях км)
Итерация 400, лучшая длина = 54 (в сотнях км)
Итерация 450, лучшая длина = 51 (в сотнях км)
Итерация 500, лучшая длина = 51 (в сотнях км)
Лучший маршрут (муравьиный алгоритм):
Вавилон -> Суса -> Византий -> Троя -> Афины -> Спарта -> Карфаген
 -> Александрия -> Мемфис -> Тир -> Иерусалим -> Ниневия -> Вавилон
Длина цикла (муравьиный алгоритм): 51.00 (в сотнях км) = 5100.00 км
Запуск полного перебора на всём графе...
Лучший маршрут (полный перебор):
Вавилон -> Ниневия -> Иерусалим -> Тир -> Мемфис -> Александрия -> Ка
 рфаген -> Афины -> Спарта -> Троя -> Византий -> Суса -> Вавилон
Длина цикла (полный перебор): 51.00 (в сотнях км) = 5100.00 км
```

Листинг 3.1 — Результат работы программы

Вывод

Результат работы алгоритма муравья совпадает с результатом работы алгоритма полного перебора за исключением самого маршрута. Программа успешно прошла тестирование на работоспособность.

4 Исследовательская часть

В данном разделе выполнена оценка трудоемкости описанных алгоритмов и проведен анализ зависимости результата работы алгоритма муравья от изменений его параметров.

4.1 Оценка трудоемкости алгоритмов

Для выполения оценки трудоемкости разработанных алгоритмов используются схемы, приведённые в разделе 2. Ниже приведены оценки трудоёмкости двух реализованных алгоритмов: полного перебора и муравьиного.

Алгоритм полного перебора

Алгоритм полного перебора формирует все возможные маршруты посещения городов, что приводит к факториальной трудоёмкости. Основные этапы алгоритма обладают следующими оценками.

1. **Инициализация:** $O(1)$ — простые операции присваивания и подготовка структур данных.
2. **Подготовка матрицы расстояний:** $O(n^2)$ — заполнение матрицы $n \times n$ посредством вычислений в теле вспомогательных функций.
3. **Формирование массива непосещённых городов:** $O(n)$ — один проход по массиву.
4. **Основной перебор:** $O(n!)$ — генерируются $(n-1)!$ перестановок (первый город фиксируется), и на каждую выполняется обработка, включающая вычисление длины маршрута за $O(n)$, что даёт итоговую оценку:

$$O(n \cdot (n-1)!) = O(n!) \tag{4.1}$$

Итоговая трудоёмкость алгоритма полного перебора:

$$T(n) = O(1) + O(n^2) + O(n) + O(n!) = O(n!) \tag{4.2}$$

Алгоритм обладает факториальной сложностью, что делает невозможным его применение при больших значениях n, однако он гарантирует получение точного оптимального решения.

Муравьиный алгоритм

Муравьиный алгоритм относится к эвристическим методам и обладает полиномиальной трудоёмкостью. Ниже приведены основные компоненты оценки.

1. **Инициализация матриц:** $O(n^2)$ — создание матриц расстояний, феромонов и эвристики размером $n \times n$ с заполнением значений для каждой пары городов.
2. **Построение маршрутов на итерации:** для каждого из m муравьёв требуется $O(n^2)$

операций (на каждом шаге маршрута просматриваются все города для вычисления вероятностей переходов). На $\mathit{iter}_{\max}$ итераций:

$$O(\mathit{iter}_{\max} \cdot m \cdot n^2). \tag{4.3}$$

3. **Испарение феромонов:** $O(n^2)$ — обновление всех элементов матрицы феромонов.
4. **Отложение феромонов:** $O(m \cdot n)$ — для каждого муравья обновляются значения феромонов на рёбрах его маршрута.

Полная оценка трудоёмкости муравьиного алгоритма:

$$T(n) = O(n^2) + O(\mathit{iter}_{\max} \cdot m \cdot n^2) + O(\mathit{iter}_{\max} \cdot n^2) + O(\mathit{iter}_{\max} \cdot m \cdot n) + O(\mathit{iter}_{\max} \cdot n) = O(\mathit{iter}_{\max} \cdot m \cdot n^2). \tag{4.4}$$

Если параметры $\mathit{iter}_{\max}$ и m фиксированы и не зависят от n, то трудоёмкость алгоритма имеет вид:

$$T(n) = O(n^2), \tag{4.5}$$

что является полиномиальной оценкой и обеспечивает возможность применения метода на больших наборах данных.

4.2 Сравнительный анализ алгоритмов

На основе проведённой оценки трудоёмкости, а также анализа достоинств и недостатков представленных методов, возможно сформулировать сравнительную характеристику алгоритма полного перебора и муравьиного алгоритма.

Алгоритм полного перебора обеспечивает нахождение строгого оптимального решения, так как просматривает все возможные перестановки городов. Однако его трудоёмкость имеет факториальный характер $O(n!)$, что приводит к крайне быстрому росту времени вычислений при увеличении числа городов. Даже при относительно небольших размерах входных данных (например, $n \geq 12$) применение данного метода становится непрактичным из-за чрезмерных вычислительных затрат. Таким образом, алгоритм полного перебора пригоден лишь для малых значений n или как эталон для проверки корректности эвристических методов.

Муравьиный алгоритм, напротив, относится к эвристическим методам и не гарантирует нахождение абсолютно оптимального решения, однако демонстрирует значительно более высокую вычислительную эффективность. Его трудоёмкость имеет полиномиальный характер $O(\mathit{iter}_{\max} \cdot m \cdot n^2)$, что позволяет применять его для существенно больших значений n. Высокая эффективность достигается за счёт:

— коллективного исследования пространства решений параллельно функционирующими муравьями;

— использования феромонной памяти, позволяющей усиливать более выгодные маршруты и постепенно подавлять неэффективные варианты;

— адаптивного управления вероятностями переходов между городами, что обеспечивает способность алгоритма изучать и эксплуатировать структуру графа.

Из результатов сравнения следует, что муравьиный алгоритм обеспечивает получение маршрутов с длиной, сопоставимой с результатом полного перебора, при существенно меньших вычислительных затратах на практических размерах задачи. Он обеспечивает баланс между качеством решения и временем расчётов. Метод полного перебора гарантирует нахождение оптимального маршрута, однако его вычислительная сложность растёт факториально с увеличением числа городов, что приводит к резкому росту времени расчётов и делает применение метода ограниченным малыми размерностями.

4.3 Параметризация

В муравьином алгоритме присутствуют переменные, которые следует задать заранее (α, β, ρ, а также количество дней), и от них возможна разница в результатах работы алгоритма. Параметризация проводилась по 3 переменным (α, β, ρ) и по дням. Значения для переменных брались из множества $0,1; 0,25; 0,5; 0,75; 0,9$, а для дней — из множества $10, 25, 50, 100, 200, 400, 800$ соответственно. Для вычисления отклонений от эталонных длин брались три полносвязных графа с двенадцатью вершинами, а затем алгоритм проходил каждый граф по десять раз. После того, как алгоритм решал задачу коммивояжёра для всех трёх графов по десять раз, вычислялись полученные средние арифметические, медианные и худшие соотношения полученных ответов при данном наборе параметров с эталонным ответом. Результаты параметризации представлены в таблице В.1.

Проведённая параметризация позволяет оценить влияние коэффициентов α, β и ρ на качество решения задачи коммивояжёра. В результате анализа установлено следующее:

— при малом числе итераций (до 50) различия между наборами параметров выражены слабо, но наблюдается тенденция к ухудшению результатов при больших значениях ρ (интенсивное испарение);

— при увеличении числа итераций до 200 — 400 стабильно улучшаются показатели среднего и медианного качества. Наиболее устойчивыми оказались параметры с невысоким коэффициентом испарения $\rho = 0.1$ — 0.25;

— Высокое значение β (0.75 — 0.9) усиливает вклад эвристической информации, что ускоряет сходимость, однако при больших α возможна преждевременная фиксация маршрутов (эффект «локальной ямы»);

— влияние коэффициента α проявляется сильнее при больших итерациях. Значения $\alpha < 0.5$ обеспечивают более сбалансированное поведение алгоритма, предотвращая доминирование феромонов на ранних этапах;

— минимальные средние значения длины маршрута достигаются при сочетаниях:

$$\alpha = 0.1 — 0.25, \quad \beta = 0.75 — 0.9, \quad \rho = 0.1 — 0.25.$$

Эти комбинации обеспечивают наиболее стабильную сходимость на всех тестируемых диапазонах числа итераций. Оптимальный режим работы алгоритма достигается при числе итераций не менее 200 — 400. При необходимости повышения точности допускается увеличение до 1000 итераций.

Таким образом, результаты работы муравьиного алгоритма, близкие к эталонному, достигаются при умеренном учёте феромонов, сильной эвристике и относительно медленном испарении. При этом устойчивость и повторяемость результатов возрастает при увеличении числа итераций.

Вывод

В данном разделе был проведена оценка трудоемкости алгоритмов и их сравнение, а также параметризация муравьиного алгоритма. Были приведены соответсвующие рекомендации на основе полученных результатов этой параметризации.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы, а именно реализованы алгоритмы полного перебора и муравья для решения задачи коммивояжёра.

Также были выполнены все задачи:

1) описаны методы решения задачи коммивояжёра на основе алгоритма полного перебора и алгоритма муравья;
2) данные алгоритмы описаны схемами;
3) на основе составленных схем выполнена оценка трудоемкости алгоритмов;
4) описанные алгоритмы реализованы;
5) выполнена параметризация второго метода, рекомендованные значения параметров: $\alpha \in [0,1; 0,25]$, $\beta \in [0,75; 0,9]$, $\rho \in [0,1; 0,25]$;
6) проведен сравнительный анализ описанных алгоритмов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы современных алгоритмов 2-е дополненное издание / Дж. Макконелл // под ред. С.Кландо, доп. М.В.Ульянова. — Москва: ТЕХНОСФЕРА, 2004. — 370 с.

2. Псевдокод алгоритма в TeX [Электронный ресурс]. URL: https://pro-prof.com/forums/topic/псевдокод-алгоритма-в-tex (дата обращения: 20 ноября 2025).

3. Visual Studio Code documentation. [Электронный ресурс]. URL: https://code.visualstudio.com/docs (дата обращения: 19 ноября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```

#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <random>
#include <limits>
#include <stdexcept>
#include <iomanip>
#include <cmath>

using namespace std;

struct Graph {
  int n = 0;
  vector<string> names;
  vector<vector<double>> dist;

  bool loadFromFile(const string& filename) {
    ifstream fin(filename);
    if (!fin.is_open()) {
      cerr << "Не удалось открыть файл: " << filename << "\n";
      return false;
    }

    string line;
    if (!getline(fin, line)) {
      cerr << "Файл пустой\n";
      return false;
    }

    istringstream iss(line);
    string city;
    names.clear();
    while (iss >> city)
    names.push_back(city);

    n = (int)names.size();
    if (n == 0) {
      cerr << "В первой строке нет названий городов\n";
      return false;
    }

```

```
    dist.assign(n, vector<double>(n, 0.0));

    for (int i = 0; i < n; ++i) {
      if (!getline(fin, line)) {
        cerr << "Недостаточно строк для матрицы расстояний\n";
        return false;
      }
      istringstream row(line);
      for (int j = 0; j < n; ++j) {
        if (!(row >> dist[i][j])) {
          cerr << "Ошибка чтения элемента d[" << i << "][" << j << "]\n";
          return false;
        }
      }
    }
    return true;
  }
};

struct Ant {
  vector<int>  tour;
  vector<bool> visited;
  double       length = 0.0;
};

double computeQ(const Graph& graph) {
  double sum = 0.0;
  int n = graph.n;
  for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
  sum += graph.dist[i][j];
  return sum / (n - 1);
}

double tourLength(const Graph& graph, const vector<int>& tour) {
  double L = 0.0;
  int n = graph.n;
  for (int i = 0; i < n - 1; i++)
  L += graph.dist[tour[i]][tour[i + 1]];
  L += graph.dist[tour.back()][tour.front()];
  return L;
}

void runAntColony(const Graph& graph, int numAnts, int maxIter, double alpha
    , double beta, double rho, vector<int>& bestTour, double& bestLength,
    bool printProgress = true) {
  int n = graph.n;
```

```
const double tau0 = 1.0;
mt19937 gen(random_device{}());
uniform_real_distribution<> urand(0.0, 1.0);
uniform_int_distribution<> startDist(0, n - 1);
vector<vector<double>> pheromone(n, vector<double>(n, tau0));
double Q = computeQ(graph);
bestLength = numeric_limits<double>::infinity();
bestTour.clear();

for (int it = 0; it < maxIter; ++it) {
  vector<Ant> ants(numAnts);

  for (int k = 0; k < numAnts; ++k) {
    Ant& ant = ants[k];
    ant.tour.assign(n, -1);
    ant.visited.assign(n, false);
    int start = startDist(gen);
    ant.tour[0] = start;
    ant.visited[start] = true;

    for (int step = 1; step < n; ++step) {
      int current = ant.tour[step - 1];

      vector<double> probs(n, 0.0);
      double sumProb = 0.0;

      for (int j = 0; j < n; ++j) {
        if (!ant.visited[j]) {
          double tau = pheromone[current][j];
          double eta = 1.0 / graph.dist[current][j];
          double val = pow(tau, alpha) * pow(eta, beta);
          probs[j] = val;
          sumProb += val;
        }
      }

      double r = urand(gen) * sumProb;
      double acc = 0.0;
      int next = -1;

      for (int j = 0; j < n; j++) {
        if (!ant.visited[j]) {
          acc += probs[j];
          if (acc >= r) {
            next = j;
            break;
          }
```

```
          }
        }

        if (next == -1) {
          for (int j = 0; j < n; j++) {
            if (!ant.visited[j]) {
              next = j;
              break;
            }
          }
        }

        ant.tour[step] = next;
        ant.visited[next] = true;
      }

      ant.length = tourLength(graph, ant.tour);

      if (ant.length < bestLength) {
        bestLength = ant.length;
        bestTour   = ant.tour;
      }
    }

    for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
    pheromone[i][j] *= (1.0 - rho);

    for (int k = 0; k < numAnts; k++) {
      const Ant& ant = ants[k];
      double delta = Q / ant.length;

      for (int idx = 0; idx < n - 1; idx++) {
        int i = ant.tour[idx];
        int j = ant.tour[idx + 1];
        pheromone[i][j] += delta;
        pheromone[j][i] += delta;
      }
      int last  = ant.tour[n - 1];
      int first = ant.tour[0];
      pheromone[last][first] += delta;
      pheromone[first][last] += delta;
    }

    if (printProgress && ((it + 1) % 50 == 0 || it == maxIter - 1))
    cout << "Итерация " << (it + 1)
    << ", текущая лучшая длина = " << bestLength
```

```
    << " (в сотнях км)\n";
  }
}


void bruteForceDFS(const Graph& graph, int startCity, vector<bool>& used,
    vector<int>& currentTour, int depth, double currentLength, vector<int>&
    bestTour, double& bestLength){
  int n = graph.n;

  if (depth == n) {
    int last = currentTour[depth - 1];
    double total = currentLength + graph.dist[last][startCity];
    if (total < bestLength) {
      bestLength = total;
      bestTour = currentTour;
    }
    return;
  }

  int lastCity = currentTour[depth - 1];

  for (int city = 0; city < n; city++) {
    if (used[city]) continue;

    used[city] = true;
    currentTour[depth] = city;

    double newLen = currentLength + graph.dist[lastCity][city];

    bruteForceDFS(graph, startCity, used, currentTour, depth + 1, newLen,
    bestTour, bestLength);

    used[city] = false;
  }
}

void runBruteForceTSP(const Graph& graph, int startCity, vector<int>&
    bestTour, double& bestLength) {
  int n = graph.n;
  bestLength = numeric_limits<double>::infinity();
  bestTour.assign(n, -1);
  vector<bool> used(n, false);
  vector<int>  currentTour(n, -1);
  used[startCity] = true;
  currentTour[0] = startCity;
  bruteForceDFS(graph, startCity, used, currentTour, /*depth=*/1, /*
```

```
    currentLength=*/0.0, bestTour, bestLength);
}

int main() {
  setlocale(LC_ALL, "Russian");

  Graph graph;
  if (!graph.loadFromFile("cities.txt")) {
    cerr << "Ошибка загрузки файла cities.txt\n";
    return 1;
  }

  cout << "Загружено городов: " << graph.n << "\n\n";

  double alpha = 0.3;   // влияние феромона
  double beta  = 0.7;   // влияние эвристики
  double rho   = 0.5;   // испарение феромона
  int numAnts = 40;
  int    maxIter = 1000;

  cout << "Запуск муравьиного алгоритма...\n";
  vector<int> bestAntTour;
  double      bestAntLen;

  runAntColony(graph, numAnts, maxIter, alpha, beta, rho, bestAntTour,
   bestAntLen);

  cout << "\nЛучший маршрут (муравьиный алгоритм):\n";
  for (int i = 0; i < (int)bestAntTour.size(); i++)
  cout << graph.names[bestAntTour[i]] << " -> ";
  cout << graph.names[bestAntTour[0]] << "\n";

  cout << fixed << setprecision(2) << "Длина цикла (муравьиный алгоритм): "
   << bestAntLen << " (в сотнях км) = " << bestAntLen * 100 << " км\n\n";

  char choice;
  cout << "Выполнить решение задачи полным перебором? (y/n): ";
  if (!(cin >> choice)) return 0;

  if (choice == 'y' || choice == 'Y') {
    const int maxBruteN = 12;

    if (graph.n > maxBruteN) {
      cout << "\nПредупреждение: n = " << graph.n << ". Полный перебор выпол
   ним только на подграфе из " << maxBruteN << " первых городов.\n\n";

      Graph small;
```

```
      small.n = maxBruteN;
      small.names.assign(graph.names.begin(), graph.names.begin() + maxBruteN);
      small.dist.assign(maxBruteN, vector<double>(maxBruteN, 0.0));
      for (int i = 0; i < maxBruteN; i++)
      for (int j = 0; j < maxBruteN; j++)
      small.dist[i][j] = graph.dist[i][j];

      cout << "Запуск полного перебора на подграфе...\n";

      vector<int> bestBFtour;
      double      bestBFlen;

      runBruteForceTSP(small, 0, bestBFtour, bestBFlen);

      cout << "\nЛучший маршрут (полный перебор, подграф):\n";
      for (int i = 0; i < (int)bestBFtour.size(); i++)
      cout << small.names[bestBFtour[i]] << " -> ";
      cout << small.names[bestBFtour[0]] << "\n";

      cout << "Длина цикла (полный перебор, подграф): " << bestBFlen << " (в сотнях км) = " << bestBFlen * 100 << " км\n";
    } else {
      cout << "\nЗапуск полного перебора на всём графе...\n";

      vector<int> bestBFtour;
      double      bestBFlen;

      runBruteForceTSP(graph, 0, bestBFtour, bestBFlen);

      cout << "\nЛучший маршрут (полный перебор):\n";
      for (int i = 0; i < (int)bestBFtour.size(); ++i)
      cout << graph.names[bestBFtour[i]] << " -> ";
      cout << graph.names[bestBFtour[0]] << "\n";

      cout << "Длина цикла (полный перебор): " << bestBFlen << " (в сотнях км) = " << bestBFlen * 100 << " км\n";
    }
  }

  return 0;
}
```

Листинг А.1 — Программная реализация описанных алгоритмов

ПРИЛОЖЕНИЕ В

В таблице В.1 приведены результаты параметризации.

Таблица В.1 — Результаты параметризации муравьиного алгоритма

α	β	ρ	**Итерация**	**Среднее**	**Медиана**	**Худшее**
0.10	0.90	0.10	10	9.20	7.50	15.00
0.10	0.90	0.10	25	4.80	5.00	7.00
0.10	0.90	0.10	50	4.60	4.50	9.00
0.10	0.90	0.10	100	3.10	3.50	6.00
0.10	0.90	0.10	200	1.50	1.50	3.00
0.10	0.90	0.10	400	1.70	2.00	4.00
0.10	0.90	0.10	800	0.90	0.00	3.00
0.10	0.90	0.25	10	9.20	9.00	14.00
0.10	0.90	0.25	25	5.90	5.50	10.00
0.10	0.90	0.25	50	3.40	3.50	7.00
0.10	0.90	0.25	100	2.40	3.00	4.00
0.10	0.90	0.25	200	1.60	2.00	2.00
0.10	0.90	0.25	400	1.10	1.00	3.00
0.10	0.90	0.25	800	0.80	0.50	2.00
0.10	0.90	0.50	10	11.00	11.50	14.00
0.10	0.90	0.50	25	5.60	5.00	10.00
0.10	0.90	0.50	50	3.60	3.00	6.00
0.10	0.90	0.50	100	2.80	3.00	6.00
0.10	0.90	0.50	200	2.30	2.00	5.00
0.10	0.90	0.50	400	1.00	0.50	4.00
0.10	0.90	0.50	800	0.30	0.00	2.00
0.10	0.90	0.75	10	8.80	8.00	14.00
0.10	0.90	0.75	25	5.40	5.00	9.00
0.10	0.90	0.75	50	4.20	4.50	8.00
0.10	0.90	0.75	100	2.20	2.50	6.00
0.10	0.90	0.75	200	2.40	3.00	4.00
0.10	0.90	0.75	400	1.30	1.50	2.00
0.10	0.90	0.75	800	0.50	0.00	2.00
0.10	0.90	0.90	10	7.20	7.50	12.00
0.10	0.90	0.90	25	5.00	6.00	7.00
0.10	0.90	0.90	50	3.50	3.50	7.00
0.10	0.90	0.90	100	3.30	4.00	5.00

Таблица B.1 — Продолжение

α	β	ρ	**Итерация**	**Среднее**	**Медиана**	**Худшее**
0.10	0.90	0.90	200	1.80	1.50	5.00
0.10	0.90	0.90	400	1.10	1.00	3.00
0.10	0.90	0.90	800	0.60	0.00	2.00
0.25	0.75	0.10	10	7.00	7.00	15.00
0.25	0.75	0.10	25	6.40	7.00	10.00
0.25	0.75	0.10	50	4.60	4.50	8.00
0.25	0.75	0.10	100	3.80	3.00	7.00
0.25	0.75	0.10	200	2.10	2.00	5.00
0.25	0.75	0.10	400	0.70	0.00	3.00
0.25	0.75	0.10	800	0.60	0.00	2.00
0.25	0.75	0.25	10	8.30	10.00	14.00
0.25	0.75	0.25	25	7.40	7.50	13.00
0.25	0.75	0.25	50	3.80	4.00	6.00
0.25	0.75	0.25	100	2.90	3.50	6.00
0.25	0.75	0.25	200	2.60	2.50	4.00
0.25	0.75	0.25	400	1.20	1.50	3.00
0.25	0.75	0.25	800	1.00	1.00	3.00
0.25	0.75	0.50	10	7.90	8.00	13.00
0.25	0.75	0.50	25	6.00	6.50	10.00
0.25	0.75	0.50	50	5.40	5.50	8.00
0.25	0.75	0.50	100	3.20	3.50	6.00
0.25	0.75	0.50	200	2.30	2.00	4.00
0.25	0.75	0.50	400	1.60	2.00	3.00
0.25	0.75	0.50	800	0.50	0.00	2.00
0.25	0.75	0.75	10	8.50	9.00	13.00
0.25	0.75	0.75	25	6.60	6.50	11.00
0.25	0.75	0.75	50	3.70	3.50	6.00
0.25	0.75	0.75	100	3.10	3.50	5.00
0.25	0.75	0.75	200	2.20	2.00	5.00
0.25	0.75	0.75	400	1.30	1.50	4.00
0.25	0.75	0.75	800	1.00	1.00	3.00
0.25	0.75	0.90	10	8.50	9.00	13.00
0.25	0.75	0.90	25	6.60	7.00	10.00
0.25	0.75	0.90	50	5.50	5.50	8.00
0.25	0.75	0.90	100	3.90	4.00	6.00

Таблица B.1 — Продолжение

α	β	ρ	**Итерация**	**Среднее**	**Медиана**	**Худшее**
0.25	0.75	0.90	200	2.30	2.50	4.00
0.25	0.75	0.90	400	1.20	1.50	3.00
0.25	0.75	0.90	800	1.00	1.00	2.00
0.50	0.50	0.10	10	10.50	11.50	15.00
0.50	0.50	0.10	25	7.40	7.00	13.00
0.50	0.50	0.10	50	6.20	6.00	12.00
0.50	0.50	0.10	100	3.40	3.50	6.00
0.50	0.50	0.10	200	3.00	2.50	5.00
0.50	0.50	0.10	400	1.50	2.00	2.00
0.50	0.50	0.10	800	1.20	1.50	2.00
0.50	0.50	0.25	10	12.10	12.00	16.00
0.50	0.50	0.25	25	7.60	8.00	12.00
0.50	0.50	0.25	50	5.60	6.00	9.00
0.50	0.50	0.25	100	4.80	5.00	7.00
0.50	0.50	0.25	200	2.80	3.00	6.00
0.50	0.50	0.25	400	2.80	3.00	5.00
0.50	0.50	0.25	800	2.50	2.50	4.00
0.50	0.50	0.50	10	9.80	9.50	14.00
0.50	0.50	0.50	25	7.40	7.50	11.00
0.50	0.50	0.50	50	5.80	6.00	10.00
0.50	0.50	0.50	100	3.90	4.00	6.00
0.50	0.50	0.50	200	3.90	4.00	6.00
0.50	0.50	0.50	400	2.50	2.50	4.00
0.50	0.50	0.50	800	2.40	2.00	4.00
0.50	0.50	0.75	10	10.10	11.50	16.00
0.50	0.50	0.75	25	7.20	7.00	13.00
0.50	0.50	0.75	50	6.20	6.50	9.00
0.50	0.50	0.75	100	4.60	5.00	7.00
0.50	0.50	0.75	200	3.80	4.00	6.00
0.50	0.50	0.75	400	3.00	3.00	5.00
0.50	0.50	0.75	800	2.00	2.00	3.00
0.50	0.50	0.90	10	10.20	11.00	16.00
0.50	0.50	0.90	25	6.60	7.00	12.00
0.50	0.50	0.90	50	6.00	6.50	10.00
0.50	0.50	0.90	100	3.90	4.00	6.00

Таблица В.1 — Продолжение

α	β	ρ	**Итерация**	**Среднее**	**Медиана**	**Худшее**
0.50	0.50	0.90	200	3.30	3.50	5.00
0.50	0.50	0.90	400	2.70	2.50	4.00
0.50	0.50	0.90	800	2.20	2.00	4.00
0.75	0.25	0.10	10	14.60	14.50	20.00
0.75	0.25	0.10	25	11.30	12.50	15.00
0.75	0.25	0.10	50	7.40	6.00	13.00
0.75	0.25	0.10	100	5.90	6.50	11.00
0.75	0.25	0.10	200	3.80	4.00	8.00
0.75	0.25	0.10	400	2.50	2.50	6.00
0.75	0.25	0.10	800	2.10	2.00	4.00
0.75	0.25	0.25	10	13.60	13.50	23.00
0.75	0.25	0.25	25	10.70	10.50	18.00
0.75	0.25	0.25	50	6.90	7.50	13.00
0.75	0.25	0.25	100	6.00	6.00	12.00
0.75	0.25	0.25	200	4.00	4.50	8.00
0.75	0.25	0.25	400	2.10	2.50	5.00
0.75	0.25	0.25	800	2.30	2.00	5.00
0.75	0.25	0.50	10	14.10	15.50	17.00
0.75	0.25	0.50	25	9.70	10.00	14.00
0.75	0.25	0.50	50	7.40	7.50	12.00
0.75	0.25	0.50	100	5.40	5.50	9.00
0.75	0.25	0.50	200	4.70	4.50	8.00
0.75	0.25	0.50	400	3.00	3.50	7.00
0.75	0.25	0.50	800	2.30	2.50	6.00
0.75	0.25	0.75	10	13.50	14.00	21.00
0.75	0.25	0.75	25	10.50	10.50	16.00
0.75	0.25	0.75	50	6.70	7.00	13.00
0.75	0.25	0.75	100	5.40	5.50	11.00
0.75	0.25	0.75	200	4.00	4.00	9.00
0.75	0.25	0.75	400	3.10	3.50	7.00
0.75	0.25	0.75	800	2.30	2.50	6.00
0.75	0.25	0.90	10	13.20	14.00	23.00
0.75	0.25	0.90	25	10.50	10.50	17.00
0.75	0.25	0.90	50	7.40	7.50	13.00
0.75	0.25	0.90	100	6.20	6.50	11.00

Таблица B.1 — Продолжение

α	β	ρ	**Итерация**	**Среднее**	**Медиана**	**Худшее**
0.75	0.25	0.90	200	4.30	4.50	8.00
0.75	0.25	0.90	400	3.10	3.00	7.00
0.75	0.25	0.90	800	2.30	2.50	6.00
0.90	0.10	0.10	10	17.10	18.00	23.00
0.90	0.10	0.10	25	13.30	14.00	21.00
0.90	0.10	0.10	50	12.10	12.50	15.00
0.90	0.10	0.10	100	10.30	10.50	14.00
0.90	0.10	0.10	200	7.60	8.00	12.00
0.90	0.10	0.10	400	5.60	5.50	9.00
0.90	0.10	0.10	800	3.30	3.00	6.00
0.90	0.10	0.25	10	17.50	18.00	23.00
0.90	0.10	0.25	25	13.70	14.00	21.00
0.90	0.10	0.25	50	11.60	12.00	15.00
0.90	0.10	0.25	100	9.20	9.50	14.00
0.90	0.10	0.25	200	7.10	7.50	12.00
0.90	0.10	0.25	400	5.50	5.50	9.00
0.90	0.10	0.25	800	3.70	4.00	7.00
0.90	0.10	0.50	10	19.30	20.00	23.00
0.90	0.10	0.50	25	14.60	15.00	20.00
0.90	0.10	0.50	50	12.00	12.50	17.00
0.90	0.10	0.50	100	10.20	10.50	14.00
0.90	0.10	0.50	200	8.20	8.50	12.00
0.90	0.10	0.50	400	6.30	6.00	10.00
0.90	0.10	0.50	800	4.90	5.00	8.00
0.90	0.10	0.75	10	18.60	18.50	27.00
0.90	0.10	0.75	25	14.60	15.00	21.00
0.90	0.10	0.75	50	12.60	13.00	17.00
0.90	0.10	0.75	100	10.70	11.00	16.00
0.90	0.10	0.75	200	8.80	9.00	13.00
0.90	0.10	0.75	400	7.20	7.50	11.00
0.90	0.10	0.75	800	5.40	5.00	9.00
0.90	0.10	0.90	10	18.60	18.50	24.00
0.90	0.10	0.90	25	15.50	16.00	22.00
0.90	0.10	0.90	50	13.30	13.50	18.00
0.90	0.10	0.90	100	11.40	11.50	17.00

Таблица B.1 — Продолжение

α	β	ρ	**Итерация**	**Среднее**	**Медиана**	**Худшее**
0.90	0.10	0.90	200	9.30	9.50	14.00
0.90	0.10	0.90	400	7.60	7.50	12.00
0.90	0.10	0.90	800	6.30	6.00	10.00